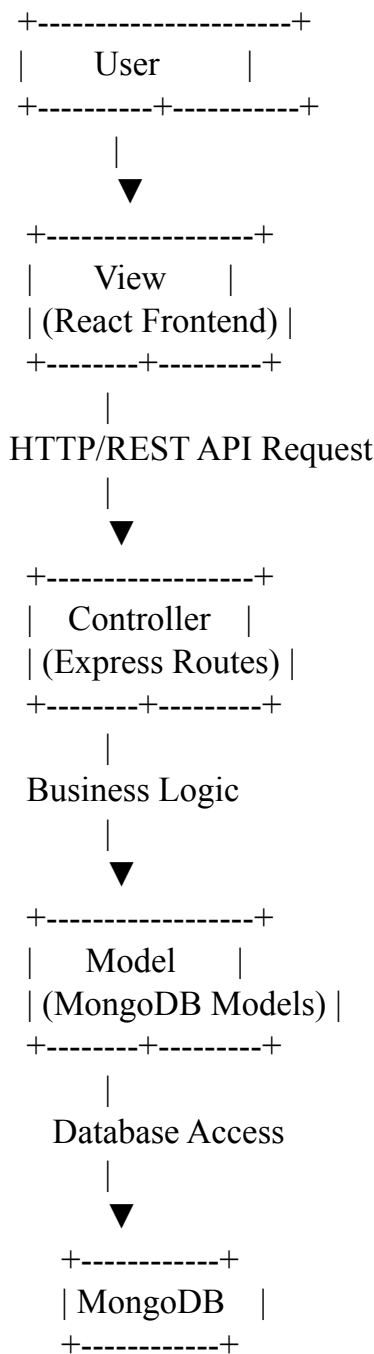


MVC (Model–View–Controller) Pattern

Overview

DarshanEase follows the **Model–View–Controller (MVC)** architectural pattern, which separates the application into three independent components: **Model**, **View**, and **Controller**. This separation improves code organization, maintainability, scalability, and simplifies future development.

MVC Architecture



1. Model

The **Model** represents the application's data and business rules. It interacts directly with the MongoDB database to store, retrieve, update, and delete information.

Responsibilities

- Store user information.
- Manage temple details.
- Store darshan slots.
- Handle booking records.
- Validate data before saving.
- Communicate with the database.

Example Models

- User Model
- Temple Model
- Slot Model
- Booking Model
- Admin Model

2. View

The **View** is the user interface presented to devotees and administrators. It displays data received from the controller and sends user actions back to the controller.

Technologies Used

- React.js
- TypeScript
- HTML5
- CSS3
- Tailwind CSS

Responsibilities

- Display temples.
- Show available darshan slots.
- Display booking forms.
- Show booking confirmations.
- Display booking history.
- Render the admin dashboard.

3. Controller

The **Controller** acts as an intermediary between the View and the Model. It receives user requests, processes business logic, communicates with the Model, and returns appropriate responses to the View.

Responsibilities

- Authenticate users.
- Validate login credentials.
- Process ticket bookings.
- Manage temple information.
- Handle booking cancellations.
- Generate booking confirmations.
- Manage users and reports.

MVC Workflow

Step 1

The user interacts with the **View** by selecting a temple or booking a darshan slot.



Step 2

The **View** sends an HTTP request to the **Controller**.



Step 3

The **Controller** validates the request and executes the required business logic.



Step 4

The **Controller** communicates with the **Model**.



Step 5

The **Model** performs database operations using MongoDB.



Step 6

The **Model** returns the requested data to the **Controller**.



Step 7

The **Controller** sends the processed response back to the **View**.



Step 8

The **View** updates the user interface with the latest information.

Advantages of MVC

- Separation of concerns between UI, business logic, and data.
- Easier code maintenance and debugging.
- Improved scalability for future enhancements.
- Better code reusability.
- Supports parallel development by frontend and backend teams.

- Simplifies testing of individual components.
- Provides a clean and organized project structure.

MVC Mapping in DarshanEase

MVC Component	Technology	Purpose
Model	MongoDB, Mongoose	Stores and manages application data
View	React.js, TypeScript, Tailwind CSS	Provides the user interface
Controller	Node.js, Express.js	Processes requests, applies business logic, and communicates with the Model

Conclusion

The MVC architecture enables DarshanEase to maintain a clear separation between the user interface, business logic, and data layer. This design improves maintainability, enhances scalability, and allows the application to evolve with additional features such as online payments, notifications, QR code verification, and mobile support without major changes to the existing codebase.